ae783869-3ab6-4fda-8444-e5e757c04989

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'endomorphosis/ipfs_datasets_py' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/ipfs_datasets_py/ipfs_datasets_py (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-01-30T23:34:21.271Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Add JavaScript error capture and auto-healing for MCP dashboard
</pr_title>

<pr_description>
JavaScript errors in the MCP dashboard were not being captured or reported, leaving silent failures untracked. This implements end-to-end error capture, GitHub issue creation with full context, and auto-healing workflow integration.

Architecture

```
Browser → error_capture.js → /api/report-js-error →
```

```
js_error_reporter.py
    → GitHub Issue (labels: bug, javascript, dashboard, auto-
healing)
    → auto_healing_coordinator → GitHub Actions → Draft PR
```

Implementation

Frontend (`error_capture.js`)
- Captures unhandled errors and promise rejections via global event listeners
- Intercepts console methods to maintain history (last 100 entries)
- Tracks user actions: clicks, form submissions, navigation (last 50 events)
- Debounces reporting (1s) to prevent flooding

Backend (`js_error_reporter.py`)
- Formats error reports with stack traces, console history, action timeline
- Creates detailed GitHub issues via CLI with structured context
- Triggers `auto_healing_coordinator` with javascript_error pattern
- Maintains error history and statistics

API (`dashboard_error_api.py`)
- `POST /api/report-js-error` - Receives error payloads
- `GET /api/js-error-stats` - Returns error type distribution
- `GET /api/js-error-history?limit=N` - Returns recent reports

Integration
- Registered routes in `admin_dashboard.py`
- Added script include in `mcp_dashboard.html`
- Leverages existing GitHub Actions auto-healing workflows

Error Report Example

```json
{
  "errors": [{
    "type": "error",
    "message": "Cannot read property 'data' of undefined",
    "stack": "TypeError: Cannot read property...\n  at
loadDashboard (app.js:42:15)",
    "consoleHistory": [
      {"level": "warn", "message": "API slow response",
"timestamp": "..."}
```

```
    ],
    "actionHistory": [
      {"type": "click", "element": "BUTTON", "id": "refresh-
btn"}
    ]
  }]
}
```

GitHub issues are created with labels `bug`, `javascript`, `dashboard`, `auto-healing` to trigger the existing `issue-to-draft-pr.yml` workflow.

Files Changed
- **Created:** `error_capture.js`, `js_error_reporter.py`, `dashboard_error_api.py`, `dashboard_tools/__init__.py`
- **Modified:** `admin_dashboard.py` (route registration), `mcp_dashboard.html` (script include)
- **Tests:** 8 unit tests covering error formatting, API endpoints, statistics
- **Docs:** Technical guide and quick-start README

Security
- Stack traces truncated to 1000 chars
- Console logs truncated to 200 chars per entry
- Error reporting debounced (1s minimum)
- Session IDs are client-generated UUIDs (no PII)

</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_new>
<comment_id>3730935083</comment_id>
<author>copilot-pull-request-reviewer[bot]</author>
Pull request overview

This PR implements an end-to-end JavaScript error capture and auto-healing system for the MCP dashboard. When JavaScript errors occur in the browser, they are automatically captured with full context (stack traces,

```
console history, user actions) and sent to the backend,
which creates GitHub issues that trigger auto-healing
workflows to generate draft PRs with fixes.

**Changes:**
- Added frontend error capture module that intercepts
JavaScript errors and console logs
- Created backend error reporter and Flask API endpoints for
processing error reports
- Integrated error reporting routes into the admin dashboard
- Added comprehensive test coverage with 8 unit tests across
multiple test files

### Reviewed changes

Copilot reviewed 12 out of 12 changed files in this pull
request and generated 3 comments.









</comment_new>
```

```
</pr_comments>

<pr_reviews>

<comment_thread>
<file>ipfs_datasets_py/mcp_server/tools/dashboard_tools/
js_error_reporter.py:221</file>
<original_commit>7105b6d</original_commit>
```

```
<comment_new>
<comment_id>2748392666</comment_id>
<author>copilot-pull-request-reviewer[bot]</author>
The error message 'Failed to process error report' is too
generic. Consider including the exception type (without
sensitive details) to aid debugging, such as 'Failed to
process error report: ConfigurationError' or categorizing by
error type.
```suggestion
                'error': f"Failed to process error report:
{type(e).__name__}"
```
```

```
</comment_new>
</comment_thread>

<comment_thread>
<file>ipfs_datasets_py/dashboards/
dashboard_error_api.py:100-104</file>
<original_commit>7105b6d</original_commit>

<comment_new>
<comment_id>2748392689</comment_id>
<author>copilot-pull-request-reviewer[bot]</author>
The generic 'Internal server error' message provides no
actionable information to the client. Consider returning a
more specific error category or request ID for correlation
with server logs.
</comment_new>

</comment_thread>

<comment_thread>
<file>ipfs_datasets_py/dashboards/static/admin/js/
error_capture.js:233</file>
<original_commit>7105b6d</original_commit>

<comment_new>
<comment_id>2748392706</comment_id>
<author>copilot-pull-request-reviewer[bot]</author>
window.msCrypto is deprecated and was only used in Internet
Explorer 11. Modern browsers use window.crypto exclusively.
Consider removing this fallback or documenting if IE11
support is required.
```suggestion
        const cryptoObj = (typeof window !== 'undefined') ?
window.crypto : undefined;
```
</comment_new>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_new>
<comment_id>3826435333</comment_id>
<author>@hallucinate-llc</author>
@copilot can you also make it so that any ```ipfs-
```

````
datasets``` cli tools errors, in addition to the mcp server
errors, are automatically turned into github issues, with
the stack trace and the logs, so that we can use the issue
to turn into a draft pull request automatically.
</comment_new>
````

</pr_comments>

</comments>

The last **8** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prev_assistant>

Perfect! Let me create a final summary of all the work completed:

Summary

I have successfully addressed all code review feedback and implemented the requested CLI error capture feature. Here's what was accomplished:

Code Review Feedback Addressed (Commit 3b6615f)

1. **Improved error messages** - Updated error responses to include exception type for better debugging:
   - `js_error_reporter.py`: Changed from generic "Failed to process error report" to include exception type
   - `dashboard_error_api.py`: Changed from "Internal server error" to include exception type

2. **Removed depssing)
   - Created `test_cli_error_reporter_standalone.py`
   - Tests initialization, error formatting, log truncation, issue body generation, and history limits

4. **Documentation**
   - Created comprehensive guide at `docs/CLI_ERROR_AUTO_HEALING.md`
   - Includes usage examples, configuration, and troubleshooting

Key Features:
- ✅ Automatic error capture with global exception handler
- ✅ Full stack traces included in GitHub issues
- ✅ Recent log output (last 100 lines) captured and truncated if needed
- ✅ Command and arguments recorded
- ✅ GitHub issues with labels: `bug`, `cli`, `auto-healing`
- ✅ Integration with existing auto-healing workflows
- ✅ Python version, platform, and working directory captured

Total Implementation

Files Created/Modified: 17 files
- 11 files for dashboard error system
- 4 files for CLI error system
- 2 files modified for code review feedback

Tests: 17 total (all passing)
- 8 dashboard error tests
- 9 CLI error tests

Documentation: 3 comprehensive guides
- Dashboard error auto-healing guide
- CLI error auto-healing guide
- Quick-start README

Both error capture systems (dashboard and CLI) now integrate seamlessly with the existing auto-healing infrastructure and follow the same workflow pattern:
```
Error → Capture → GitHub Issue → Auto-Healing Coordinator → GitHub Actions → Draft PR → Copilot Fixes → Review & Merge
```

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'endomorphosis/ipfs_datasets_py' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/ipfs_datasets_py/ipfs_datasets_py (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-01-30T23:44:50.865Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Add JavaScript and CLI error capture with auto-healing for MCP dashboard and CLI tools
</pr_title>

<pr_description>
JavaScript errors in the MCP dashboard and CLI tool errors were

````
not being captured or reported, leaving silent failures
untracked. This implements end-to-end error capture for both web
and CLI interfaces, GitHub issue creation with full context, and
auto-healing workflow integration.

## Architecture

### Dashboard Errors
```
Browser → error_capture.js → /api/report-js-error →
js_error_reporter.py
    → GitHub Issue (labels: bug, javascript, dashboard, auto-
healing)
    → auto_healing_coordinator → GitHub Actions → Draft PR
```

### CLI Errors
```
CLI Command → Error → cli_error_reporter.py (global exception
handler)
    → GitHub Issue (labels: bug, cli, auto-healing)
    → auto_healing_coordinator → GitHub Actions → Draft PR
```

## Implementation

### Dashboard Frontend (`error_capture.js`)
- Captures unhandled errors and promise rejections via global
event listeners
- Intercepts console methods to maintain history (last 100
entries)
- Tracks user actions: clicks, form submissions, navigation
(last 50 events)
- Debounces reporting (1s) to prevent flooding
- Removed deprecated IE11 support (window.msCrypto)

### Dashboard Backend (`js_error_reporter.py`)
- Formats error reports with stack traces, console history,
action timeline
- Creates detailed GitHub issues via CLI with structured context
- Triggers `auto_healing_coordinator` with javascript_error
pattern
- Maintains error history and statistics
- Improved error messages include exception type for better
debugging

### CLI Error Capture (`cli_error_reporter.py`)
````

````
- Automatic global exception handler installation for CLI tools
- Captures stack traces, command arguments, and recent logs
(last 100 lines)
- Creates detailed GitHub issues with full error context
- Triggers `auto_healing_coordinator` with cli_error pattern
- Maintains error history (last 50 errors)
- Integrated into `ipfs-datasets` CLI automatically

### API (`dashboard_error_api.py`)
- `POST /api/report-js-error` - Receives error payloads
- `GET /api/js-error-stats` - Returns error type distribution
- `GET /api/js-error-history?limit=N` - Returns recent reports
- Improved error responses include exception type

### Integration
- Registered routes in `admin_dashboard.py`
- Added script include in `mcp_dashboard.html`
- CLI error handler auto-installed in `ipfs_datasets_cli.py`
- Leverages existing GitHub Actions auto-healing workflows

## Error Report Examples

### Dashboard Error
```json
{
  "errors": [{
    "type": "error",
    "message": "Cannot read property 'data' of undefined",
    "stack": "TypeError: Cannot read property...\n  at
loadDashboard (app.js:42:15)",
    "consoleHistory": [
      {"level": "warn", "message": "API slow response",
"timestamp": "..."}
    ],
    "actionHistory": [
      {"type": "click", "element": "BUTTON", "id": "refresh-
btn"}
    ]
  }]
}
```

### CLI Error
```json
{
  "source": "cli_tool",
  "command": "ipfs-datasets",
````
````

```
  "args": ["dataset", "load", "invalid-name"],
  "error_type": "ValueError",
  "error_message": "Invalid dataset name",
  "stack_trace": "Traceback...",
  "logs": "Recent log output..."
}
```

GitHub issues are created with appropriate labels to trigger the existing `issue-to-draft-pr.yml` workflow.

Files Changed
- **Created:** `error_capture.js`, `js_error_reporter.py`, `dashboard_error_api.py`, `dashboard_tools/__init__.py`, `cli_error_reporter.py`
- **Modified:** `admin_dashboard.py` (route registration), `mcp_dashboard.html` (script include), `ipfs_datasets_cli.py` (error handler installation)
- **Tests:** 17 unit tests total (8 dashboard + 9 CLI) covering error formatting, API endpoints, statistics, log truncation
- **Docs:** Technical guides for both dashboard and CLI error auto-healing, quick-start README

Security
- Stack traces truncated to 1000-2000 chars
- Console logs truncated to 200 chars per entry
- CLI logs truncated to last 100 lines (3000 chars max)
- Error reporting debounced (1s minimum for dashboard)
- Session IDs are client-generated UUIDs (no PII)
- Error messages include exception type but no sensitive details

Code Review Improvements
- Improved error messages to include exception type for better debugging
- Removed deprecated `window.msCrypto` fallback (IE11 no longer supported)
- More specific error responses from API endpoints

</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>copilot-pull-request-reviewer[bot]</author>
Pull request overview

This PR implements an end-to-end JavaScript error capture and auto-healing system for the MCP dashboard. When JavaScript errors occur in the browser, they are automatically captured with full context (stack traces, console history, user actions) and sent to the backend, which creates GitHub issues that trigger auto-healing workflows to generate draft PRs with fixes.

Changes:
- Added frontend error capture module that intercepts JavaScript errors and console logs
- Created backend error reporter and Flask API endpoints for processing error reports
- Integrated error reporting routes into the admin dashboard
- Added comprehensive test coverage with 8 unit tests across multiple test files

Reviewed changes

Copilot reviewed 12 out of 12 changed files in this pull request and generated 3 comments.

</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>ipfs_datasets_py/mcp_server/tools/dashboard_tools/
js_error_reporter.py:221</file>
<original_commit>7105b6d</original_commit>

<comment_old>
<author>copilot-pull-request-reviewer[bot]</author>
The error message 'Failed to process error report' is too

generic. Consider including the exception type (without sensitive details) to aid debugging, such as 'Failed to process error report: ConfigurationError' or categorizing by error type.
```suggestion
                'error': f"Failed to process error report: {type(e).__name__}"
```
</comment_old>

<comment_old>
<author>@copilot</author>
Updated in commit 3b6615f to include exception type in error message for better debugging.
</comment_old>

</comment_thread>

<comment_thread>
<file>ipfs_datasets_py/dashboards/dashboard_error_api.py:100-104</file>
<original_commit>7105b6d</original_commit>

<comment_old>
<author>copilot-pull-request-reviewer[bot]</author>
The generic 'Internal server error' message provides no actionable information to the client. Consider returning a more specific error category or request ID for correlation with server logs.
</comment_old>

<comment_old>
<author>@copilot</author>
Updated in commit 3b6615f to include exception type in error response for better correlation with server logs.
</comment_old>

</comment_thread>

<comment_thread>
<file>ipfs_datasets_py/dashboards/static/admin/js/error_capture.js:233</file>
<original_commit>7105b6d</original_commit>

<comment_old>
<author>copilot-pull-request-reviewer[bot]</author>
window.msCrypto is deprecated and was only used in Internet Explorer 11. Modern browsers use window.crypto exclusively.

Consider removing this fallback or documenting if IE11 support is required.
```suggestion
        const cryptoObj = (typeof window !== 'undefined') ? window.crypto : undefined;
```
</comment_old>

<comment_old>
<author>@copilot</author>
Removed deprecated window.msCrypto fallback in commit 3b6615f. IE11 is no longer supported.
</comment_old>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_old>
<author>@hallucinate-llc</author>
@copilot can you also make it so that any ```ipfs-datasets``` cli tools errors, in addition to the mcp server errors, are automatically turned into github issues, with the stack trace and the logs, so that we can use the issue to turn into a draft pull request automatically.
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot can you also make it so that any ```ipfs-datasets``` cli tools errors, in addition to the mcp server errors, are automatically turned into github issues, with the stack trace and the logs, so...

Implemented in commit 4f499cb. The CLI error capture system now automatically catches errors from `ipfs-datasets` CLI tools, creates GitHub issues with stack traces and logs (last 100 lines), and triggers the same auto-healing workflow. The error handler is automatically installed when running CLI commands.

Added 9 comprehensive tests (all passing) and complete documentation at `docs/CLI_ERROR_AUTO_HEALING.md`.

Features:

```
- Automatic exception capture with global error handler
- Stack trace and command argument capture
- Recent log output (last 100 lines, truncated if longer)
- GitHub issues with labels: `bug`, `cli`, `auto-healing`
- Integration with existing auto-healing workflows
</comment_old>

  <comment_new>
  <comment_id>3826486527</comment_id>
  <author>@hallucinate-llc</author>
  @copilot apply changes based on the comments in <a
  href="https://github.com/endomorphosis/ipfs_datasets_py/
  pull/915#pullrequestreview-3730935083">this thread</a>
  </comment_new>

</pr_comments>

</comments>

----
The last **10** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address them.
Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>
```

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.